Electron Desktop Application Setup Guide

Converting Drone-A-to-B-Waypoint-Nav-System to Windows (.exe)

Project: Drone-A-to-B-Waypoint-Nav-System
Frontend: React + Vite
Target: Windows Desktop Application (.exe)

---

1. Prerequisites

Install the following software:

- Node.js 20.x
- npm
- Git
- Existing Vite React project working correctly

Verify:

```
node -v
npm -v
```

Example

```
Node v20.x.x
npm 10.x.x
```

---

2. Install Electron Dependencies

From the project root:

```
npm install -D electron@^41 electron-builder concurrently wait-on cross-env
```

These packages are used for:

Package	Purpose
electron	Desktop application runtime
electron-builder	Creates Windows installer (.exe)
concurrently	Runs Vite and Electron together
wait-on	Waits until Vite starts
cross-env	Cross-platform environment variables

---

3. Modify package.json

Add Main Entry

```
"main": "electron/main.cjs"
```

Add Scripts

```
"scripts": {
  "electron:dev": "concurrently -k \"cross-env BROWSER=none vite\" \"wait-on
http://localhost:5173 && cross-env ELECTRON_START_URL=http://localhost:5173
electron .\"",
  "electron:build": "vite build && electron-builder --win"
}
```

Add Build Configuration

```
"build": {
  "appId": "com.drone.dashboard",
  "productName": "Drone Mission Dashboard",

  "files": [
    "dist/**/*",
    "electron/**/*"
  ],

  "directories": {
```

```
    "output": "release"
  },

  "win": {
    "target": "nsis",
    "icon": "assets/icon.ico"
  }
}
```

Add Override

```
"overrides": {
  "app-builder-lib": {
    "@noble/hashes": "^1.8.0"
  }
}
```

Run:

```
npm install
```

---

4. Create Electron Folder

Create:

```
electron/
```

Inside it create:

```
electron/
    main.cjs
    preload.cjs
```

---

5. main.cjs

```
const { app, BrowserWindow } = require("electron");
const path = require("path");

function createWindow() {

    const win = new BrowserWindow({

        width: 1400,
        height: 900,

        webPreferences: {

            preload: path.join(__dirname, "preload.cjs"),

            contextIsolation: true,

            nodeIntegration: false

        }

    });

    const devUrl = process.env.ELECTRON_START_URL;

    if (devUrl) {

        win.loadURL(devUrl);

    } else {

        win.loadFile(path.join(__dirname, "..", "dist", "index.html"));

    }

}

app.whenReady().then(createWindow);

app.on("window-all-closed", () => {

    if (process.platform !== "darwin")

        app.quit();
```

```
});
```

---

6. preload.cjs

```
// Security stub
```

Nothing else is required.

---

7. Modify Vite Configuration

Open

```
vite.config.ts
```

Add

```
base: './'
```

Example

```
export default defineConfig({

    base: './',

    plugins: [
        react()
    ]

})
```

Without this the packaged application usually opens a white screen because assets are referenced with absolute paths.

---

8. Backend Configuration

Create

```
src/backendUrl.ts
```

```
const KEY = "backendUrl";

export function getBackendUrl() {

    return localStorage.getItem(KEY) || "http://localhost:8000";

}

export function setBackendUrl(url: string) {

    localStorage.setItem(KEY, url);

}
```

---

9. Update API

Instead of

```
const API_URL = "http://localhost:8000";
```

Use

```
import { getBackendUrl } from "./backendUrl";

const API_URL = getBackendUrl();
```

Now the desktop app can connect to another computer running ROS2.

Example

```
http://192.168.1.120:8000
```

instead of

```
localhost
```

---

10. Backend Settings Page (Optional but Recommended)

Create

```
src/components/BackendSettings.tsx
```

Purpose:

- Input backend IP
- Save button
- Store using localStorage
- No rebuild required

---

11. App Icon

Create

```
assets/
    icon.ico
```

Recommended

```
256x256
```

Multi-resolution Windows icon.

---

12. Project Structure

```
Drone-A-to-B-Waypoint-Nav-System

│
├── assets
│      └── icon.ico
│
├── electron
│      ├── main.cjs
│      └── preload.cjs
│
├── src
│      ├── backendUrl.ts
│      ├── api.ts
│      └── components
│              └── BackendSettings.tsx
│
├── package.json
├── vite.config.ts
│
└── dist
```

---

13. Development Mode

Run

```
npm run electron:dev
```

This will

- Start Vite
- Wait for Vite
- Launch Electron
- Enable hot reload

---

14. Production Build

Run

```
npm run electron:build
```

Electron Builder will

- Build Vite
- Package Electron
- Create Installer

Output

```
release/
```

Example

```
release/

Drone Mission Dashboard Setup 0.1.0.exe
```

---

15. Installation

Double-click

```
Drone Mission Dashboard Setup 0.1.0.exe
```

Install like any Windows application.

Launch

```
Drone Mission Dashboard
```

---

16. Connecting to ROS2 Backend

If backend runs on same PC

```
http://localhost:8000
```

If backend runs on Raspberry Pi

```
http://192.168.x.x:8000
```

If backend runs on Jetson

```
http://192.168.x.x:8000
```

No rebuild required.

---

17. Files Added

New files

```
electron/main.cjs
electron/preload.cjs
src/backendUrl.ts
src/components/BackendSettings.tsx
assets/icon.ico
```

---

18. Existing Files Modified

```
package.json
vite.config.ts
src/api.ts
```

---

19. Build Output

```
release/

Drone Mission Dashboard Setup.exe
```

This is the distributable Windows application that can be installed on any Windows computer.

---

20. Summary

New Files

- electron/main.cjs
- electron/preload.cjs
- src/backendUrl.ts
- src/components/BackendSettings.tsx
- assets/icon.ico

Modified Files

- package.json
- vite.config.ts
- src/api.ts

Commands

Install dependencies

```
npm install -D electron@^41 electron-builder concurrently wait-on cross-env
```

Development

```
npm run electron:dev
```

Production

```
npm run electron:build
```

Installer Output

```
release/
    Drone Mission Dashboard Setup.exe
```

The Electron wrapper does **not** modify the ROS2 backend or the drone logic. It simply packages the existing Vite frontend into a native Windows desktop application while continuing to communicate with the same backend API.